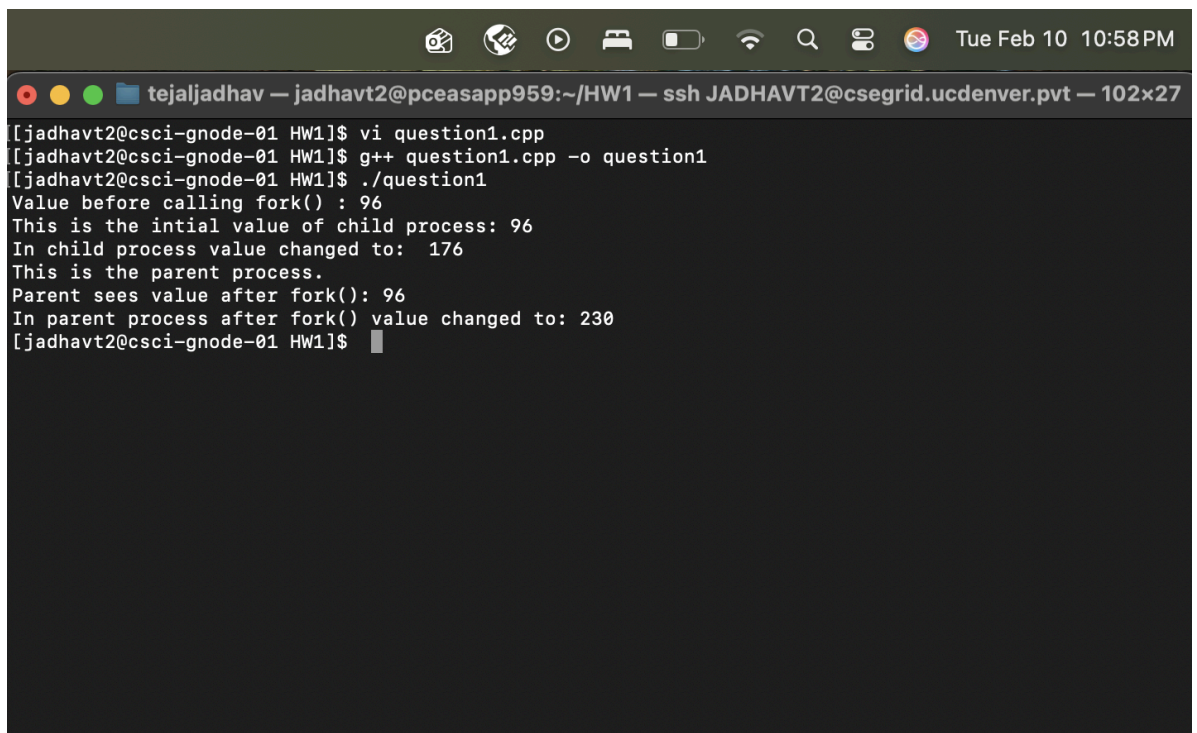


Homework Assignment 1

1. Write a program that calls `fork()`. Before calling `fork()`, have the main process access a variable (e.g., `x`) and set its value to something (e.g., 100). What value is the variable in the child process? What happens to the variable when both the child and parent change the value of `x`?

Output Screenshot -



```
tejaljadhav — jadhav2@pceasapp959:~/HW1 — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[jadhav2@csci-gnode-01 HW1]$ vi question1.cpp
[jadhav2@csci-gnode-01 HW1]$ g++ question1.cpp -o question1
[jadhav2@csci-gnode-01 HW1]$ ./question1
Value before calling fork() : 96
This is the intial value of child process: 96
In child process value changed to: 176
This is the parent process.
Parent sees value after fork(): 96
In parent process after fork() value changed to: 230
[jadhav2@csci-gnode-01 HW1]$
```

What value is the variable in the child process? What happens to the variable when both the child and parent change the value of `x`?

Answer-

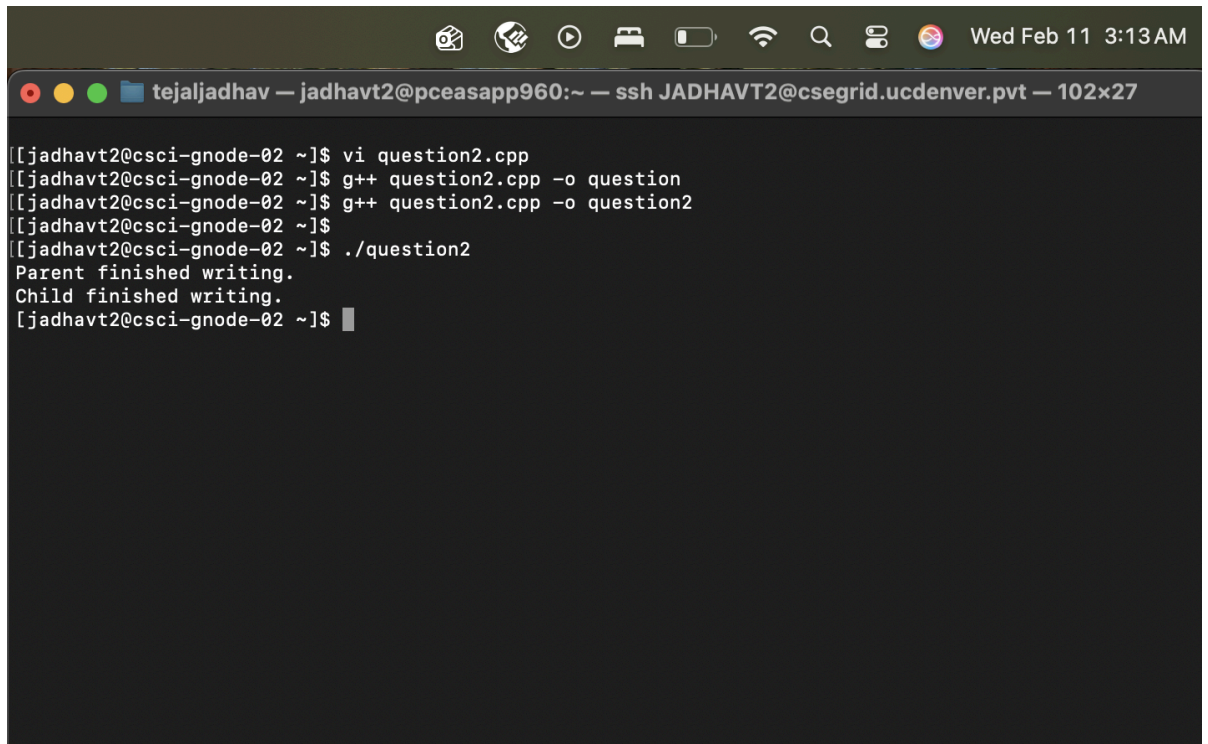
The value of the variable in the child process is 96, which is the value assigned before calling `fork()`. When `fork()` is executed, the child gets a copy of the parent's memory, so it starts with the same value.

After that, when the child changes the value to 176, it only changes its own copy. The parent's value is not affected. Similarly, when the parent changes the value to 230, it only affects the parent's copy.

This shows that after `fork()`, the parent and child processes have separate memory spaces, so changes made in one process do not affect the other.

2. Write a program that opens a file (with the `open()` system call) and then calls `fork()` to create a new process. Can both the child and parent access the file descriptor returned by `open()`? What happens when they are writing to the file concurrently, i.e., simultaneously?

Output Screenshot -



```
[[jadhav2@csci-gnode-02 ~]$ vi question2.cpp
[[jadhav2@csci-gnode-02 ~]$ g++ question2.cpp -o question
[[jadhav2@csci-gnode-02 ~]$ g++ question2.cpp -o question2
[[jadhav2@csci-gnode-02 ~]$
[[jadhav2@csci-gnode-02 ~]$ ./question2
Parent finished writing.
Child finished writing.
[[jadhav2@csci-gnode-02 ~]$
```

Can both the child and parent access the file descriptor returned by `open()`? What happens when they are writing to the file concurrently, i.e., simultaneously?

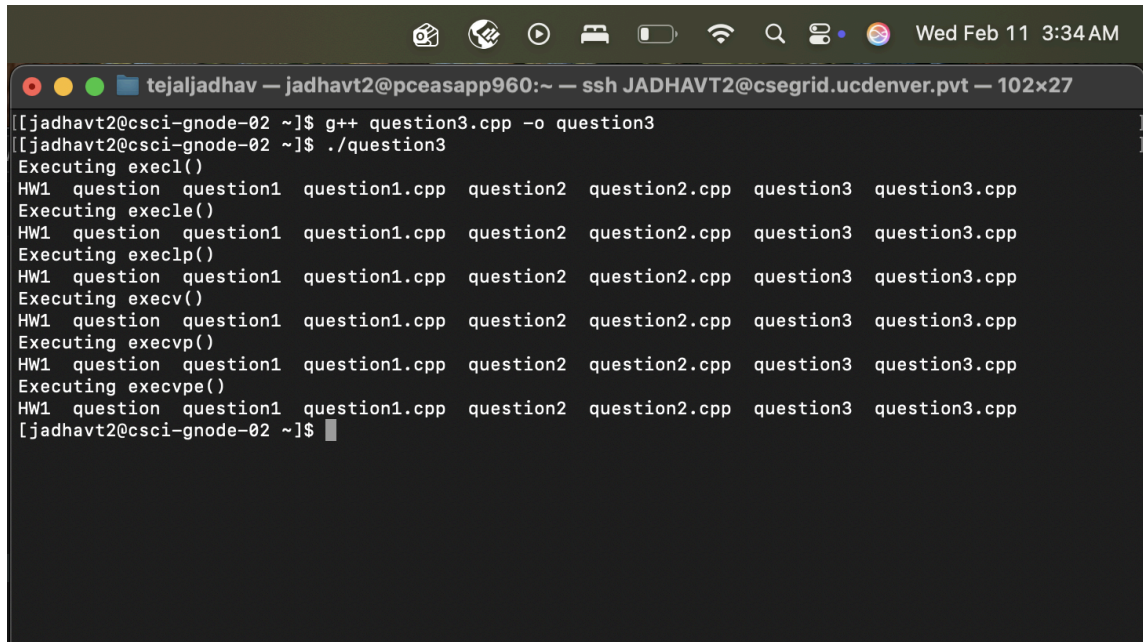
Answer-

Yes, both the parent and child can use the file descriptor returned by `open()`. After `fork()` is called, the child gets a copy of the file descriptor from the parent.

Both processes still refer to the same file in the system, so they share the same file offset. If they write to the file at the same time, the order of writing is not guaranteed. The output in the file may appear in different order depending on which process runs first.

3. Write a program that calls `fork()` and then calls some form of `exec()` to run the program `/bin/ls`. See if you can try all of the variants of `exec()`, including (on Linux) `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()`, and `execvpe()`. Why do you think there are so many variants of the same basic call?

Output Screenshot -



```
tejaljadhav — jadhav2@pceasapp960:~ — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[jadhav2@csci-gnode-02 ~]$ g++ question3.cpp -o question3
[jadhav2@csci-gnode-02 ~]$ ./question3
Executing execl()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
Executing execle()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
Executing execlp()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
Executing execv()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
Executing execvp()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
Executing execvpe()
HW1 question question1 question1.cpp question2 question2.cpp question3 question3.cpp
[jadhav2@csci-gnode-02 ~]$
```

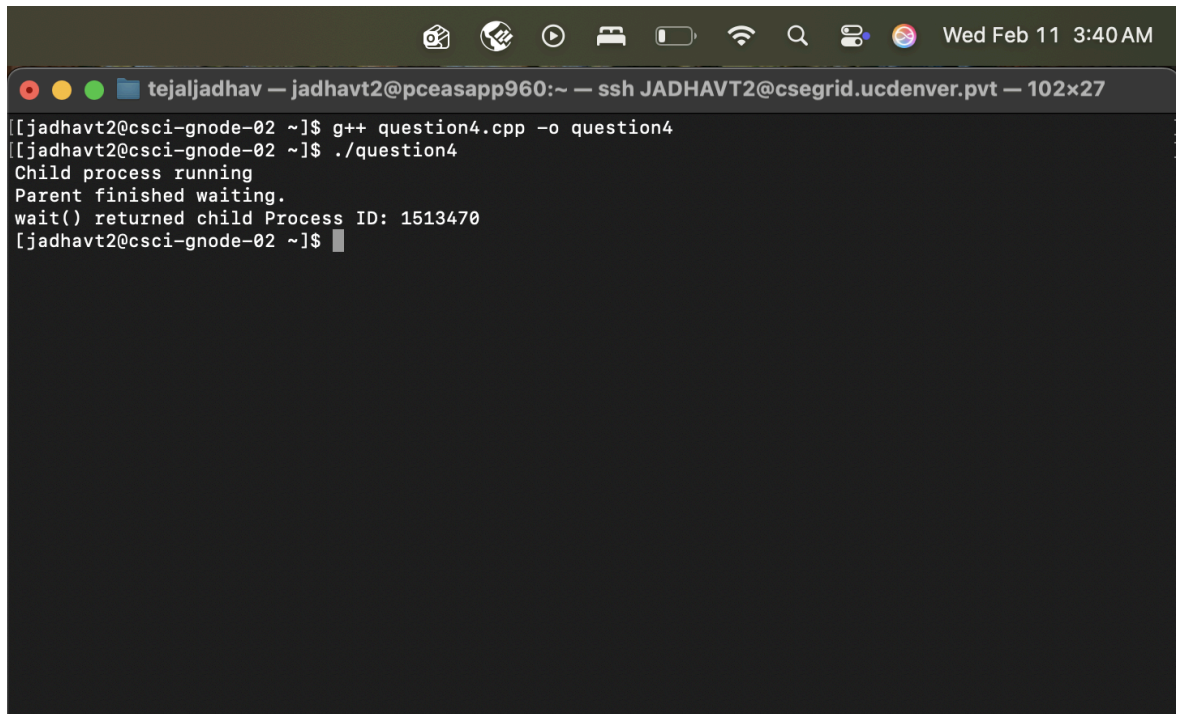
Why do you think there are so many variants of the same basic call?

Answer-

There are many versions of `exec()` because they allow different ways to pass arguments and environment variables to a program. Some versions take arguments as a list, while others take them as an array. The versions ending with “p” search the `PATH` variable, so we don’t need to give the full path of the program. The versions ending with “e” allow us to pass a custom environment. These different versions make `exec()` more flexible to use in different situations.

4. Now write a program that uses `wait()` to wait for the child process to finish in the parent. What does `wait()` return? What happens if you use `wait()` in the child?

Output Screenshot -



```
tejaljadhav — jadhavt2@pceasapp960:~ — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[[jadhavt2@csci-gnode-02 ~]$ g++ question4.cpp -o question4
[[jadhavt2@csci-gnode-02 ~]$ ./question4
Child process running
Parent finished waiting.
wait() returned child Process ID: 1513470
[[jadhavt2@csci-gnode-02 ~]$
```

What does `wait()` return?

Answer-

`wait()` returns the process ID (PID) of the child process that has finished execution.

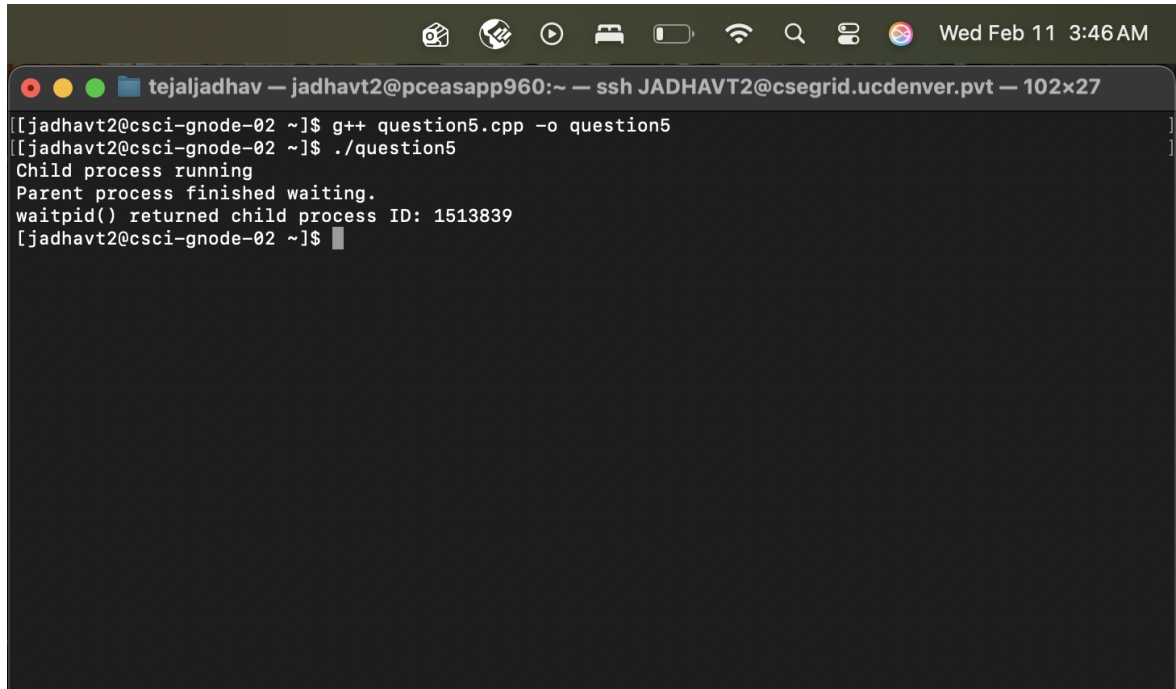
What happens if you use `wait()` in the child?

Answer-

If `wait()` is used in the child process, it returns -1 because the child does not have any child processes to wait for.

5. Write a slight modification of the previous program, this time using `waitpid()` instead of `wait()`. When would `waitpid()` be useful?

Output Screenshot -



```
tejaljadhav — jadhav2@pceasapp960:~ — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[[jadhav2@csci-gnode-02 ~]$ g++ question5.cpp -o question5
[[jadhav2@csci-gnode-02 ~]$ ./question5
Child process running
Parent process finished waiting.
waitpid() returned child process ID: 1513839
[[jadhav2@csci-gnode-02 ~]$
```

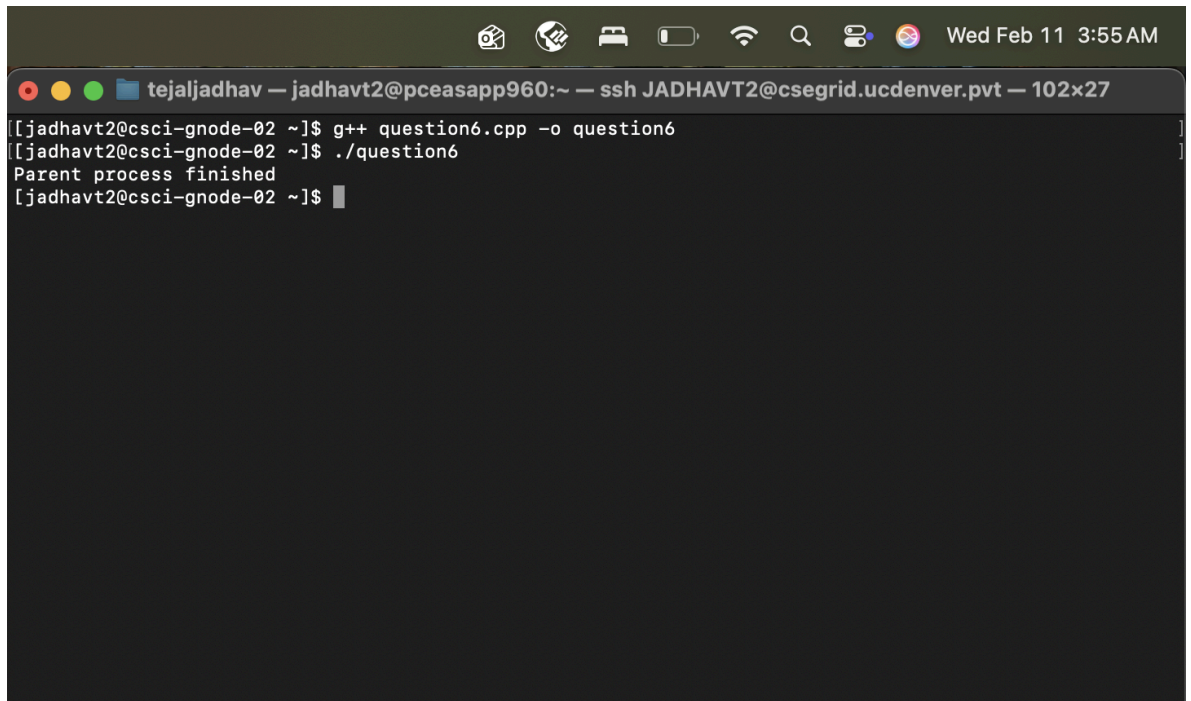
When would `waitpid()` be useful?

Answer-

`waitpid()` is useful when we want to wait for a specific child process instead of waiting for any child. It gives more control to the parent process, especially when there are multiple child processes. It can also be used with options to check the child's status without blocking.

6. Write a program that creates a child process, and then in the child closes standard output (STDOUT_FILENO). What happens if the child calls printf() to print some output after closing the descriptor?

Output Screenshot -



```
tejaljadhav — jadhav2@pceasapp960:~ — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[[jadhav2@csci-gnode-02 ~]$ g++ question6.cpp -o question6
[[jadhav2@csci-gnode-02 ~]$ ./question6
Parent process finished
[[jadhav2@csci-gnode-02 ~]$
```

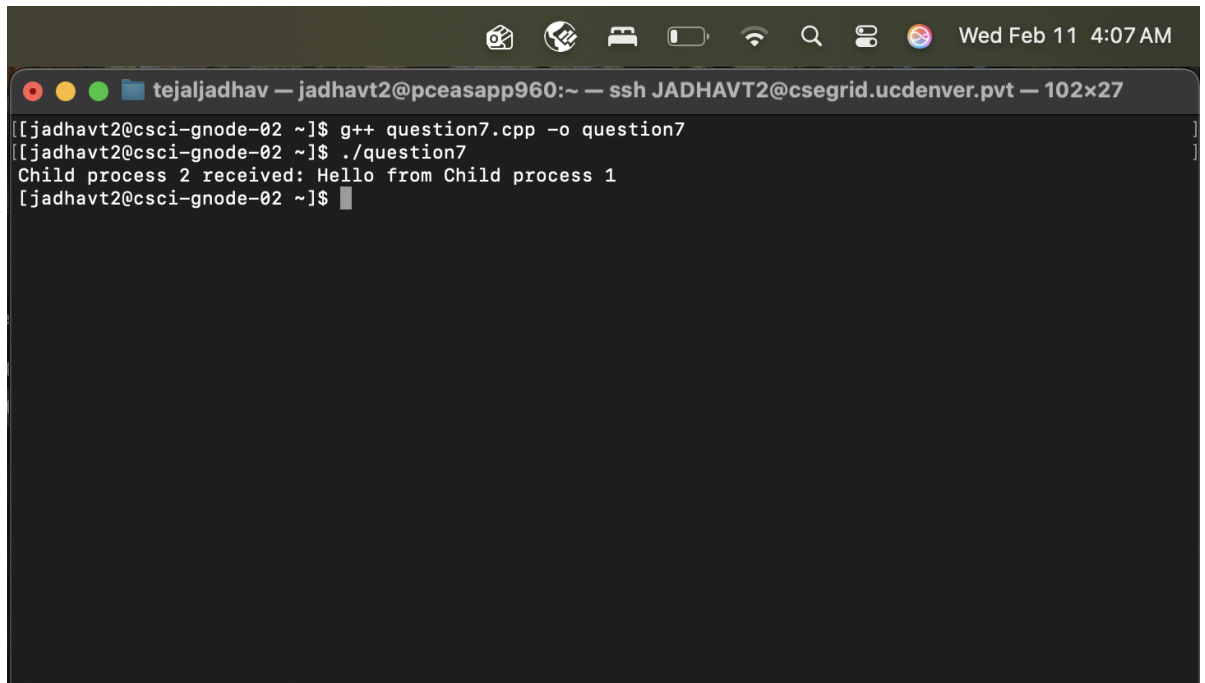
What happens if the child calls printf() to print some output after closing the descriptor?

Answer-

If the child closes `STDOUT_FILENO` and then calls `printf()`, nothing will be printed. This is because standard output is closed, so the child has no output stream to write to. The parent process is not affected and can still print normally.

7. Write a program that creates two children and connects the standard output of one to the standard input of the other using the pipe() system call.

Output Screenshot -



```
tejaljadhav — jadhvt2@pceasapp960:~ — ssh JADHAVT2@csegrid.ucdenver.pvt — 102x27
[[jadhvt2@csci-gnode-02 ~]$ g++ question7.cpp -o question7
[[jadhvt2@csci-gnode-02 ~]$ ./question7
Child process 2 received: Hello from Child process 1
[[jadhvt2@csci-gnode-02 ~]$
```

This program creates two child processes and connects them using a pipe. The first child sends a message through the pipe by writing to its standard output. The second child reads that message from its standard input. This shows how one process's output can become another process's input using pipe().